Rapport de stratégie : "LesLoosers"

Lilou DURET, Pierre-Léo DUCRUET, Adodo Joyce LAMA

14/05/2025

Introduction

Pour commencer ce projet, nous avons, chacun, pris le temps de lire attentivement les consignes. L'objectif était de bien comprendre les règles du jeu et de réfléchir à une première stratégie. Comme nos emplois du temps ne coïncidaient pas toujours, nous avons d'abord dû travailler en décalé.

Phase 1 : Analyse des règles et préparation

Nous avons commencé par établir un tableau récapitulatif listant les coups possibles et les gains associés. Cela nous a permis d'identifier plus facilement les coups potentiellement intéressants.

Grâce à cette analyse, nous avons constaté que le coup "5" était à éviter : il permettait de ne gagner que 5 points, alors que si l'adversaire jouait "4", iel remportait 9 points. Le risque nous a semblé trop élevé, donc nous l'avons mis de côté dans un premier temps.

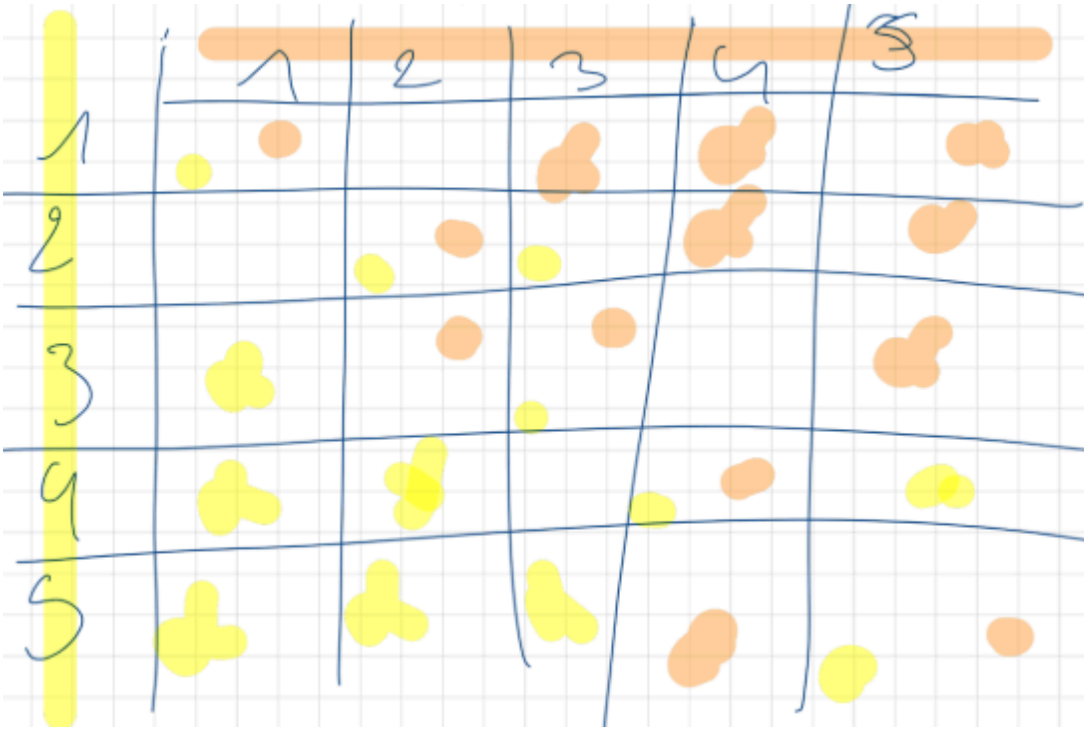

FIGURE 1 : TABLEAU RÉCAPITULATIF

Parallèlement, nous avons commencé à imaginer des stratégies alternatives, différentes de celles proposées dans le programme de M. Lemaire. Nous avons appelé ces variantes des "stratégies *lambda"* : des stratégies simples que nous pensions que d'autres équipes pouvaient concevoir. L'idée était de pouvoir les anticiper ou les contrer.

Cependant, nous avons vite réalisé qu'il existait des stratégies presque impossibles à contrer, notamment la stratégie random. Même si elle ne garantit pas une victoire, elle est très difficile à anticiper. Nous avons donc décidé de noter cette stratégie comme à haut risque, et d'élaborer au moins un plan pour éviter une défaite totale face à elle.

Correction de notre interprétation initiale

Au début, nous avions mal compris les consignes : nous pensions pouvoir choisir librement la version du jeu à utiliser. Nous nous sommes donc d'abord concentrés uniquement sur la version 1, ce qui a influencé notre premier développement. Une fois la consigne clarifiée, nous avons réajusté notre approche.

Nous savions dès le départ que notre stratégie devait s'adapter au fil du temps. Une stratégie statique aurait été, selon nous, une erreur. Nous nous sommes donc mis d'accord : notre stratégie finale comporterait plusieurs étapes, chacune dépendant de l'historique des coups joués par l'adversaire.

Construction d'une première stratégie

Face à l'incertitude des choix des autres équipes, nous avons d'abord envisagé une stratégie de secours : un programme très simple qui jouait uniquement le coup "1", garantissant ainsi un gain minimal. Cela permettait de ne pas finir avec 0 point.

Mais nous avons vite compris que ce type de stratégie était trop défensive. Elle assurait la survie, mais pas la victoire, et elle ne reflétait pas nos compétences en Prolog.

Nous avons alors adopté une approche plus nuancée, intégrant un système de pondération basé sur les coups précédents de l'adversaire. L'objectif était de détecter ses préférences pour mieux orienter nos choix. Une simple fréquence ne suffisait pas, car sur 100 tours, les données pouvaient rester floues. Nous avons donc imaginé un système où chaque coup joué augmentait un poids associé, ce qui faisait émerger plus rapidement les tendances.

Mise en place d'une stratégie évolutive

Nous avons décomposé notre stratégie en plusieurs phases :

- Tours 1 à 10 : nous jouions systématiquement le coup 4.
- Tours 11 à 20 : nous passions en mode aléatoire (*random*).

Ce choix avait pour but de brouiller les pondérations adverses, en partant du principe que beaucoup de nos camarades allaient utiliser un système basé sur la fréquence des coups joués. Ces 20 premiers coups nous servaient donc de *leurrage*.

Ensuite, notre stratégie s'adaptait selon les observations :

1. Nous analysions les coups adverses grâce à notre pondération.
2. Si cette analyse révélait un coup favori, nous adoptions une stratégie d'imitation (nommée *Imitation -1*) : nous jouions systématiquement ce même coup pendant 30 tours, ce qui nous permettait de maximiser nos gains grâce au système d'addition des points.
3. En parallèle, la pondération continuait à se mettre à jour.

4. Si après un certain temps aucun coup favori ne se dégageait, cela signifiait très probablement que l'adversaire utilisait une stratégie random ou qu'iel a brusquement changé de stratégie.
5. Dans ce cas, nous basculions aussi en mode aléatoire, mais avec une touche personnalisée : tous les 8 tours, nous glissions un coup "4", afin de garder un certain potentiel de score élevé jusqu'au tour 80.

Suite à cela, pendant les 15 rounds suivants, nous jouerons le coup "4", puis, pour les 5 derniers rounds, le coup "1".Cette dernière partie est une touche personnelle, au cas où la victoire se jouerait à quelques points près

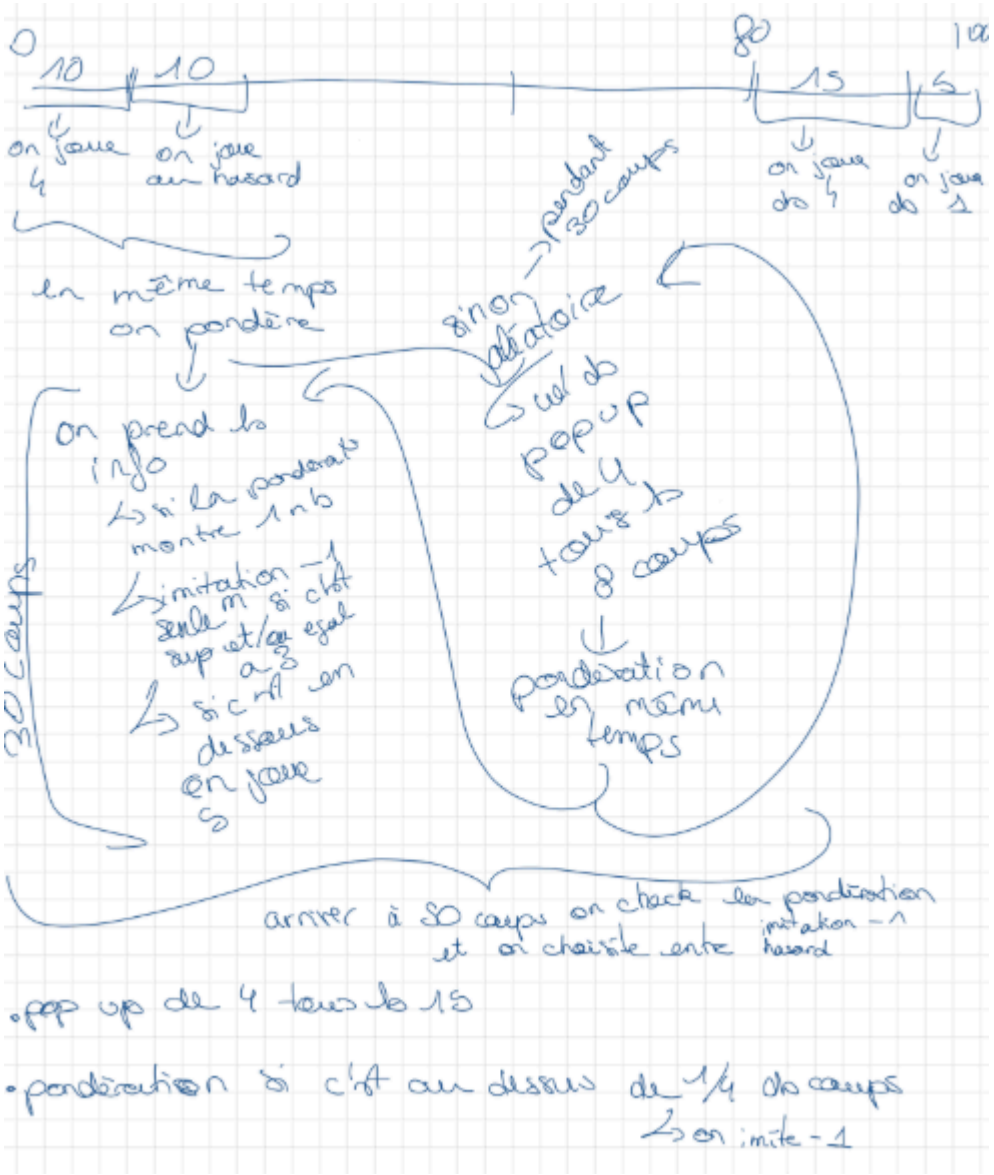

FIGURE 2 : NOTRE PREMIÈRE STRATÉGIE

Élaboration de la deuxième stratégie

Au fil du projet, nous avons expérimenté plusieurs pistes pour établir une stratégie gagnante. Parmi elles, l'une des premières idées consistait à jouer de manière aléatoire avec une forte préférence pour le coup "4", car c'est celui qui rapporte le plus de points dans la majorité des cas. Toutefois, cette méthode s'est révélée trop risquée : si l'adversaire anticipait cette tendance, elle devenait facilement contrable.

- **Phase 1 : 5 coups de "4" pour déstabiliser**

Notre stratégie finale débute chaque partie avec 5 coups de "4". Cette ouverture à comme but :

- Déstabiliser l'adversaire en jouant un coup de manière consécutive
- Brouiller les statistiques des adversaires
- Collecter des données sur les réponses de l'adversaire, dans le but d'analyser son comportement
- Gagner des points dès le début si possible

```
joue(lesLoosers,L, 4) :-
        length(L, N), N =< 10, !, pondération(L).
```

Nous n'adaptons pas encore notre réponse, mais nous analysons déjà l'adversaire via :

```
pondération(L)
```

```
pondération(L) :-
        coupsAdversaire(L, ListeCoups),
        frequences(ListeCoups, Frequences),
        write('Stats adversaire : '), write(Frequences), nl.
```

- **Phase 2 : Pondération et détection des tendances**

Passé le 11e tour, notre stratégie devient adaptive.

```
joue(lesLoosers,Coupsprec,R):-
    (\+ faitspoidsini -> poidsini ; true),
    maj(Coupsprec),
    trouvenbmax(Max),
    (Max >1 -> R is Max-1;
    random_between(1,5,R)).
```

```
poidsini :-
    forall(between(1;5,N),assertz(poids(N,1))),
    assertz(faitspoidsini).
```

→ Initialise une pondération à 1 pour chaque coup (1 à 5), une seule fois.

```
maj([]).
maj([[_,CA]|L]):-
    retract(poids(CA,P)),
    NewP is P + 1,
    assertz(poids(CA, NewP)),
    maj(L).
```

→ À chaque tour, met à jour les poids en augmentant celui du coup joué par l'adversaire.

```
trouvenbma(Max):-
    findall(P-N, poids(N,P), Liste),
    keysort(Liste, Trie),
    last(Trie, _-Max).
```

→ Trouve le coup le plus fréquent (ayant le plus grand poids).

Si un coup est clairement dominant (Max > 1), nous jouons un contre-coup calculé via :
R is ((Max - 1) mod 5) + 1 ou R is Max-1 (nous avons modifié le code plus tard)

```
(Max >1 -> R is Max-1;
```

→ Ce calcul vise à répondre avec un coup différent de celui le plus joué par l'adversaire.

Si aucun coup n'émerge comme dominant ce qui est typique d'une stratégie aléatoire, on choisit :

```
random_between(1,5,R)).
```

→ Autrement dit, on adopte nous-mêmes une stratégie aléatoire pour tenter de limiter les pertes.

- **Phase 3 : Affichage des statistiques adverses**

```
frequence_lesloosers(Liste, [F1, F2, F3, F4, F5]) :-
nb_de_fois_lesloosers(1, Liste, F1),
nb_de_fois_lesloosers(2, Liste, F2),
nb_de_fois_lesloosers(3, Liste, F3),
nb_de_fois_lesloosers(4, Liste, F4),
nb_de_fois_lesloosers(5, Liste, F5).
nb_de_fois_lesloosers(_, [], 0).

nb_de_fois_lesloosers(X, [X|T], N) :- nb_de_fois_lesloosers(X, T, N1), N
is N1 + 1, !.
nb_de_fois_lesloosers(X, [Y|T], N) :- nb_de_fois_lesloosers(X, T, N).
```

permet d'afficher les fréquences exactes des coups joués par l'adversaire, utilisées ensuite pour calculer la stratégie de réponse. Cela alimente notre logique pondérée.

Retour du tournoi

Suite aux retours de M. Lemaire, nous avons pris le temps de revenir sur notre programme pour corriger plusieurs points soulevés. Il nous a notamment fait remarquer que notre code affichait parfois des choses inutiles et que certaines erreurs ne semblaient pas évidentes à première vue.
En retravaillant sur le programme, on s'est rendu compte qu'on avait recréé des prédicats qui existaient déjà dans Prolog, comme “max_list/2”. Ce genre de doublon n’avait aucune utilité et était une erreur de notre part. C’était une première prise de conscience, et on a donc fait en sorte de ne plus ré-écrire des fonctions standard du langage.
On a aussi supprimé les write, qui étaient là à des fins de debug, mais qui ne servaient plus. Cela a permis de rendre le programme plus propre et d’éviter des affichages inutiles pendant son exécution.
Un autre problème qu'on n’avait pas vu au départ, c’est que le fait “faitspoidsini_lesloosers” n’était jamais initialisé. Du coup, même si le programme semblait fonctionner à première vue pendant nos tests, ce détail posait des soucis dans certains cas. C’était assez frustrant à repérer, parce qu'en testant, il n'y avait pas d’erreur.
Là, on a un peu avancé à l'aveugle, ce qui n’était pas forcément la meilleure méthode. On aurait peut-être pu demander plus d’aide plus tôt, mais on voulait vraiment essayer de résoudre ce problème par nous-mêmes. Cependant, nous nous sommes vite retrouvés bloqués sans savoir exactement d'où venait le bug.
Finalement, après avoir essayé de déboguer un bon moment sans succès, on a demandé de l’aide à nos proches, et avec leurs conseils, on a pu corriger les erreurs principales. Par exemple, la définition de “frequence_lesloosers/2” ne fonctionnait pas comme prévu avec “forall/2”, ce qui empêchait de remplir correctement la liste des fréquences (avec F1, F2, F3, F4, F5).
Voici un résumé des corrections principales qu'on a apportées :

- Correction de frequence_lesloosers/2 pour qu’elle retourne correctement une liste des fréquences.
- Ajout de :- discontiguous joue/3 pour éviter les erreurs de compilation liées aux clauses non regroupées de joue/3.
- Suppression des variables singleton inutilisées, ce qui a également permis de se débarrasser des warnings du compilateur.
- Ajout d'une protection dans retract pour éviter les erreurs si un poids n’existe pas déjà.
- Une structure du programme plus claire et mieux organisée.

En résumé, après avoir pris le temps de revoir le code et de corriger ces erreurs, le programme fonctionne désormais de manière plus fluide et sans les problèmes qu’on rencontrait avant.

```
:- dynamic poids_lesloosers/2.
:- dynamic faitspoidsini_lesloosers/0.
:- discontiguous joue/3.

joue(lesloosers, L, 4) :-
    length(L, N),
    N =< 10, !,
    pondération_lesloosers(L).

joue(lesloosers, CoupsPrec, R) :-
    (\+ faitspoidsini_lesloosers -> poidsini_lesloosers ; true),
    maj_lesloosers(CoupsPrec),
    trouvenbmax_lesloosers(Max),
    (Max > 1 -> R is ((Max - 1) mod 5) + 1 ; random_between(1, 5, R)).

pondération_lesloosers(L) :-
    ca_lesloosers(L, ListeCoups),
    frequence_lesloosers(ListeCoups, _).

ca_lesloosers([], []).
ca_lesloosers([[_, Adv]|Reste], [Adv|Liste]) :-
    ca_lesloosers(Reste, Liste).

frequence_lesloosers(Liste, Frequencies) :-
    findall(F, (between(1, 5, X), nb_de_fois_lesloosers(X, Liste, F)), Frequencies).

nb_de_fois_lesloosers(_, [], 0).
nb_de_fois_lesloosers(X, [X|T], N) :-
    nb_de_fois_lesloosers(X, T, N1),
    N is N1 + 1, !.
nb_de_fois_lesloosers(X, [_|T], N) :-
    nb_de_fois_lesloosers(X, T, N).

poidsini_lesloosers :-
    forall(between(1, 5, N), assertz(poids_lesloosers(N, 1))),
    assertz(faitspoidsini_lesloosers).

maj_lesloosers([]).
maj_lesloosers([[_, CA]|L]) :-
    (   retract(poids_lesloosers(CA, P))
    ->  NewP is P + 1,
        assertz(poids_lesloosers(CA, NewP))
    ;   true      ),
    maj_lesloosers(L).

trouvenbmax_lesloosers(Max) :-
    findall(P, poids_lesloosers(_, P), ListePoids),
    max_member(Max, ListePoids).
```

Conclusion

Ce projet nous a permis de réfléchir à des stratégies à la fois dynamiques et adaptables, tout en nous donnant l'occasion de mieux comprendre et manipuler le langage Prolog. On a appris à construire une logique évolutive, à anticiper les réactions adverses, et à ajuster notre approche en fonction de ce qui se passe en cours de partie. C'était à la fois un défi technique et une vraie expérience de réflexion stratégique.

Nous avons :

- Appris à travailler à distance et en asynchrone
- Corrigé rapidement une mauvaise interprétation des consignes
- Élaboré une stratégie en plusieurs étapes conditionnelles
- Mis en place une logique pondérée pour analyser les adversaires
- Appris à utiliser "dynamic"

Cependant, nous avons été bloqués à plusieurs reprises à cause des limites de nos compétences actuelles et aussi parce que nous avions des attentes trop ambitieuses. Cela nous a poussés à chercher de l'aide extérieure à plusieurs reprises, et à utiliser certaines notions que nous n'avions pas encore vues en cours. C'est un peu dommage, mais en même temps, cela nous a permis d'apprendre comment fonctionnaient ces concepts et de mieux comprendre leur utilité dans un vrai programme.

Notre stratégie n'est pas parfaite, mais elle est capable de s'adapter à plusieurs styles de jeu. Elle reflète bien notre progression, notre collaboration, notre réflexion, et notre volonté de faire preuve d'analyse stratégique.

Annexe :

FIGURE 3 : CODE FINAL

```
:- dynamic poids_lesloosers/2.
:- dynamic faitspoidsini_lesloosers/0.
:- discontiguous joue/3.

joue(lesloosers, L, 4) :-
    length(L, N),
    N =< 10, !,
    pondération_lesloosers(L).

joue(lesloosers, CoupsPrec, R) :-
    (\+ faitspoidsini_lesloosers -> poidsini_lesloosers ; true),
    maj_lesloosers(CoupsPrec),
    trouvenbmax_lesloosers(Max),
    (Max > 1 -> R is ((Max - 1) mod 5) + 1 ; random_between(1, 5, R)).

pondération_lesloosers(L) :-
    ca_lesloosers(L, ListeCoups),
    frequence_lesloosers(ListeCoups, _).

ca_lesloosers([], []).
ca_lesloosers([[_, Adv]|Reste], [Adv|Liste]) :-
    ca_lesloosers(Reste, Liste).

frequence_lesloosers(Liste, Frequencies) :-
    findall(F, (between(1, 5, X), nb_de_fois_lesloosers(X, Liste, F)), Frequencies).

nb_de_fois_lesloosers(_, [], 0).
nb_de_fois_lesloosers(X, [X|T], N) :-
    nb_de_fois_lesloosers(X, T, N1),
    N is N1 + 1, !.
nb_de_fois_lesloosers(X, [_|T], N) :-
    nb_de_fois_lesloosers(X, T, N).

poidsini_lesloosers :-
    forall(between(1, 5, N), assertz(poids_lesloosers(N, 1))),
    assertz(faitspoidsini_lesloosers).

maj_lesloosers([]).
maj_lesloosers([[_, CA]|L]) :-
    (   retract(poids_lesloosers(CA, P))
    ->  NewP is P + 1,
        assertz(poids_lesloosers(CA, NewP))
    ;   true      ),
    maj_lesloosers(L).
```

```
trouvenbmax_lesloosers(Max) :-
    findall(P, poids_lesloosers(_, P), ListePoids),
    max_member(Max, ListePoids).
```